

函数

exit 函数

- **return 关键字**:
 - 返回当前函数调用。将返回值返回调用者。（在底层，会调用 `_exit()` 函数。）
- **exit() 函数**:
 - 退出当前程序

```
1 // 函数声明
2 // int test(int a, char ch);
3 int test(int, char); // 函数声明的简化写法。声明时，形参名可以省略。
4
5 int main(void)
6 {
7     // 函数调用
8     int ret = test(10, 'a'); // test函数，调用结束，return 给 main
9
10    printf("test函数返回: ret = %d\n", ret);
11
12    // return 0;    // 返回给调用者（启动例程）—— 作用：结束程序。
13    exit(0);       // 结束程序。
14 }
15
16 // 函数定义
17 int test(int a, char ch)
18 {
19     printf("a = %d\n", a);
20     printf("ch = %c\n", ch);
21
22     // return 97;    // 返回给调用者，程序不结束。
23     // 结束程序
24     exit(97);       // 使用 #include <stdlib.h>
25 }
```

多文件编程

- **头文件守卫**：为了防止头文件被重复包含。--- head.h
 1. `#pragma once` 是 VS 自动生成的。只应用于 windows 系统。

```
2.
1 #ifndef _HEAD_H_    // 习惯写成这样。
2 #define _HEAD_H_
3 ... 头文件内容: #include <xx.h>/宏定义 #define PI 3.14/函数声明/类型定义
4 #endif
5
6 // 示例:
7 #ifndef _HEAD_H_    // 标准引入“头文件守卫”
8 #define _HEAD_H_
9
10 // include 头文件
11 #include <stdio.h>
```

```
12 #include <string.h>
13 #include <stdlib.h>
14 #include <math.h>
15 #include <time.h>
16
17 // 函数声明
18 int add(int a, int b);
19 int sub(int a, int b);
20 // 宏定义
21 #define PI 3.14
22
23 // 类型定义
24
25 #endif
26
```

- <> 包谷的是 系统库头文件。
- "" 包裹的是，用户自定义头文件。

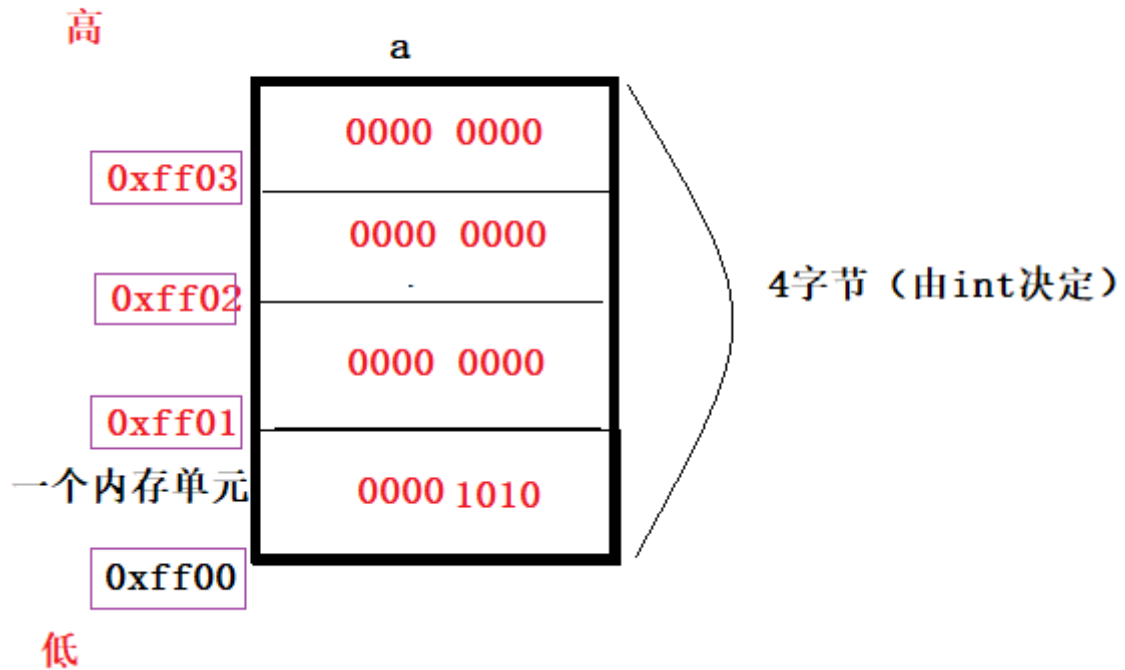
```
1 // main函数所在的 .c 文件中:
2 #include "head.h"
```

指针

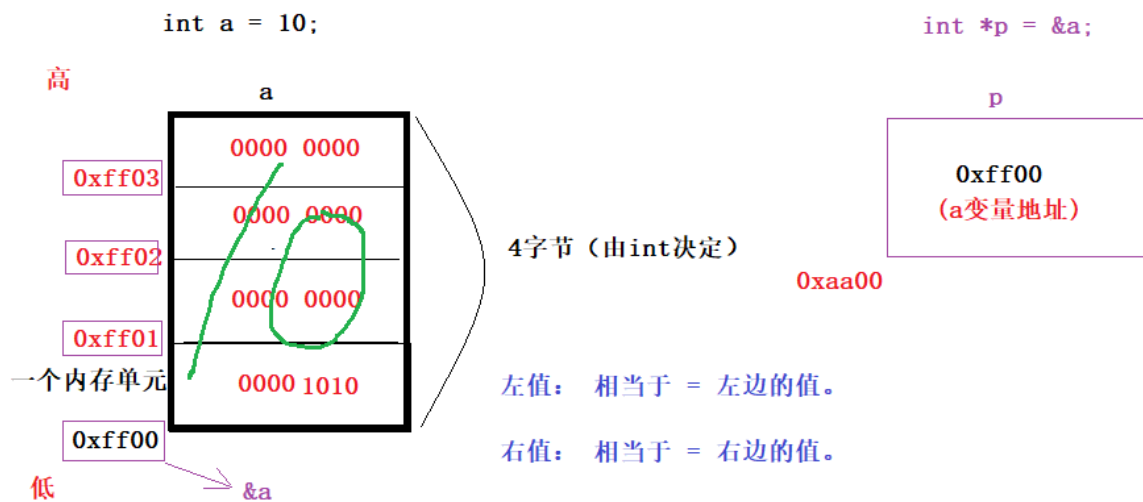
指针和内存单元

- 指针：地址！
- 指针变量：用存储地址的变量！
- 内存单元：是计算机中内存最小的存储单位。内存单元大小 —— 1字节(8bit位)。
 - 每个内存单元，都有一个唯一的编号。
 - 这个内存单元的编号，称之为“地址”

```
int a = 10;
```



指针的定义和使用



```
1 int a = 10;
2 int *p = &a; //int* p --- windows    int *p ---Linux    int * p 依据个人习惯
3 // int *p = &a; 展开:
4 // int *p;
5 // p = &a;
6 a = 999;
7 printf("*p = %d\n", *p); // 间接引用 -- 右值。
8
9 // 解引用、间接引用。
10 *p = 250; // 间接引用 -- 左值。
11 printf("a = %d\n", a);
```

- ***p 作用:**

- 将 `p` 变量的内容, 取出, 当成地址看待。找到该地址对应的内存空间。
 - 如果做左值, 存储数据到空间中。

- 如果做右值，取出空间中的内容。
- **指针类型大小**。（指针，算一种自定义数据类型。int *）
 - 指针的大小，与类型无关！只与当前使用的平台架构有关（32位：4字节、64位：8字节）。

```
1 int main(void)
2 {
3     printf("int *的大小: %u\n", sizeof(int*));
4     printf("short *的大小: %u\n", sizeof(short*));
5     printf("char *的大小: %u\n", sizeof(char*));
6     printf("long *的大小: %u\n", sizeof(long*));
7     printf("long long *的大小: %u\n", sizeof(long long *));
8 }
```

- 可以在一条语句中，同时定义多个指针变量、普通变量。

```
1 int a, b, c; // 多个普通变量
2
3 int *p1, *p2, *p3; // 多个指针变量。 每个变量都有有一个自己的 *
4
5 int a, *b, *c, d; // 定义整型变量a、d， 同时定义指针变量 b、c
```

空指针和野指针

野指针

1. 没有使用“有效”的地址，给指针初始化。

```
1 int* p; //没有给 p 指定一个有效地址。
2 *p = 1000;
```

2. p指针变量有值，但是，该值不是一个有效的地址。

```
1 int* p = 10; // 没有给 p 指定一个有效地址。
2 *p = 1000;
```

【结论】：编程时，**杜绝野指针**。

空指针

- NULL == 0 == \0

```
1 int* p = NULL; // #define NULL ((void *)0) // 定义一个空指针。
2 *p = 1000; // *p 所指向的内存空间，是一个“无效访问区域”。
```

泛型指针 (万能指针 void *)

- 可以接受任何一种变量的地址。但是，在使用时【必须】借助“强制类型转换”具体化数据类型。

- void * 类型的大小：32位：4字节。64位：8字节。

```
1  int main(void)
2  {
3      printf("void *的大小: %u\n", sizeof(void *));
4
5      char ch = 'R';
6      void* p;          // 泛型指针(万能指针)
7
8      p = &ch;
9
10     printf("*p = %c", *(char *)p); // 将p变量类型, 由void* 强转成 char *
11
12     return EXIT_SUCCESS;
13 }
```

const关键字

修饰变量

```
1  const int a = 20;    // a 为只读变量, 不能修改。
2  //a = 200;          // 不可以修改
3  int* p = &a;
4  *p = 677;
5  printf("a = %d\n", a);    // 借助指针可以修改 const 普通变量的值。
```

修饰指针

```
1  // 方式1
2  const int *p;        // 向后(右)作用
3
4  // 示例:
5  int a = 10;
6  int b = 20;
7  const int *p = &a;
8  *p = 500;            // 【失败】: 将 a 的值, 改为 500, 不能改!
9  p = &b;              // 【成功】: 可以修改 p变量的内容(地址)。
10
11 // 方式2
12 int const *p;        // 向后(右)作用
13 作用方式同上!
```

- 常用：在函数内，限制指针所指向的内存空间，为只读(不允许修改)

指针和数组

数组名

- 数组名，是地址常量。—— 不可以被修改、赋值(=、+=、-=、*=、/=、%= 带有副作用的运算符)

```

1 | int a[3] = { 1,2,3 }; // a 就是数组地址。
2 | int b[3];           // b 是常量
3 |
4 | b = a; // 不允许!! 因为 b 为地址常量。

```

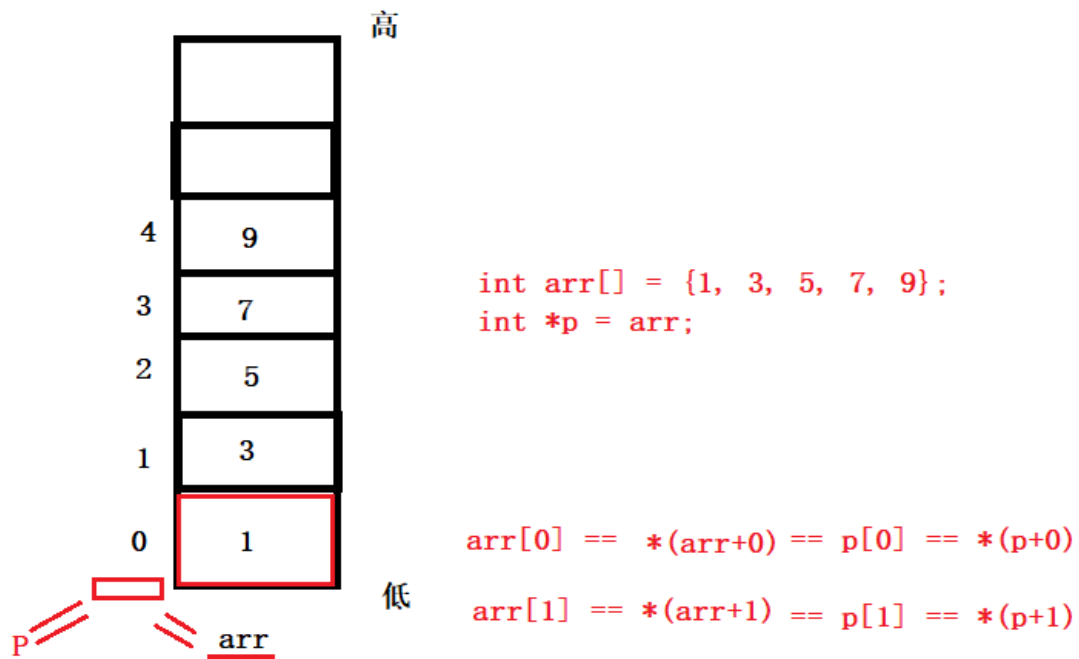
- 指针，是变量。可以使用数组名，给指针赋值。

```

1 | int *p = a; // a 就是数组地址。允许!!

```

取数组元素



```

1 | int arr[] = {1, 3, 5, 7, 9};
2 | int *p = arr; // 使用数组地址，给p指针变量初始化
3 |
4 | // 结论:
5 | arr[0] == *(arr+0) == p[0] == *(p+0)
6 | arr[1] == *(arr+1) == p[1] == *(p+1)
7 | arr[2] == *(arr+2) == p[2] == *(p+2)
8 | ...
9 | arr[N] == *(arr+N) == p[N] == *(p+N)

```

指针和数组名区别

1. 指针是变量。数组名是常量。
2. sizeof (指针) ——> 4字节、8字节。
3. sizeof (数组名) ——> 数组实际的字节数。

指针的算数运算

数据类型对指针的作用

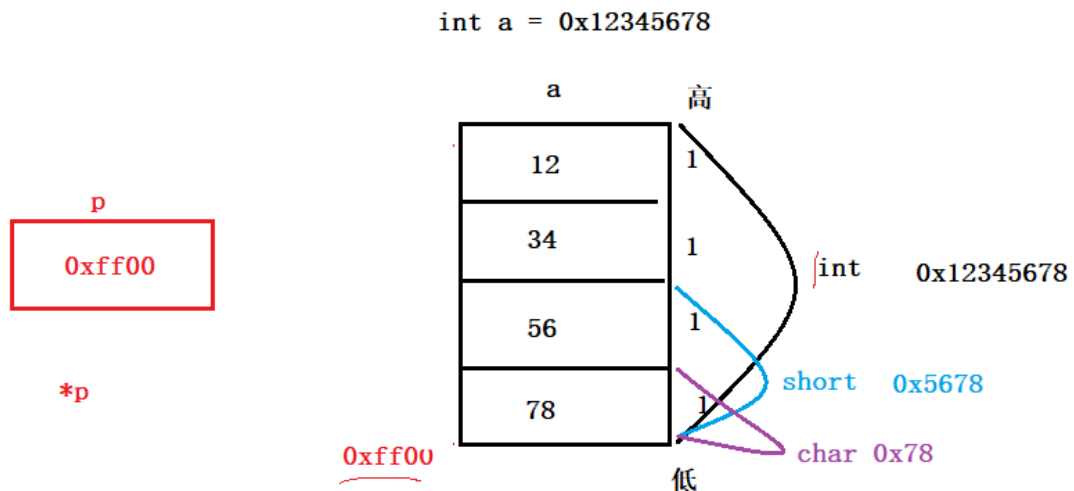
1. 间接引用 (解引用) :

- 指针的数据类型，绝定了从指针存储的地址开始，向后读取的字节数。（与指针本身存储空间无关）

```

1  int a = 0x12345678;
2
3  //int* p = &a;
4  //int* p; --- 0x12345678;
5  //short* p; --- 0x5678;
6  char* p; // --- 0x78;
7  p = &a;
8
9  printf("%#x\n", *p);

```



2. 加减运算。

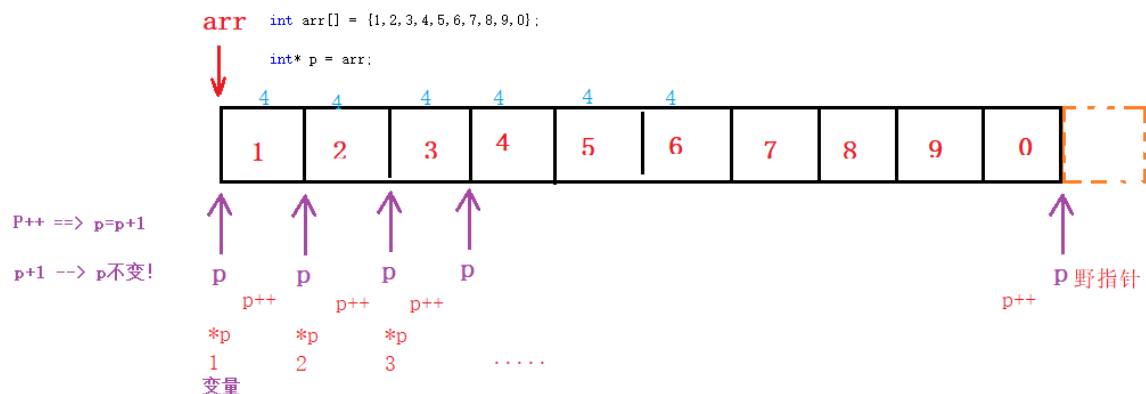
- 指针的数据类型，决定了指针进行 +/- 操作时，向后/前 跳过的字节数。

```

1  int * +1 实际加过 4 字节。
2  short * +1 实际加过 2 字节。
3  char * +1 实际加过 1 字节。
4  long long * +1 实际加过 8 字节。

```

指针++操作数组



```

1  int main(void)
2  {

```

```

3   int arr[] = {1,2,3,4,5,6,7,8,9,0};
4   int* p = arr;
5
6   int n = sizeof(arr) / sizeof(arr[0]);
7
8   //for (size_t i = 0; i < n; i++)
9   //{
10  //    printf("%d ", arr[i]);
11  //}
12
13  for (size_t i = 0; i < n; i++)
14  {
15      printf("%d ", *p);
16      p++;          // p = p+1, 一次加过 一个 int 大小（一个数组元素）。
17  }
18  // p值随着循环，不断变化， 打印结束后，p指向一块无效的内存空间（野指针）
19  printf("\n");
20
21  return 0;
22  }

```

练习

- 练习：使用 指针给空数组连续赋值。再使用“指针挪移”方法打印出这个数组。

```

1   int main(void)
2   {
3       int arr[10];
4       int n = sizeof(arr) / sizeof(arr[0]);
5       int* p = arr; // p 指向 arr[0]
6       printf("p = %p\n", p);
7
8       // 使用 指针，给空数组连续赋值。
9       for (size_t i = 0; i < n; i++)
10      {
11          //arr[i] = i + 10;
12          *(p + i) = 10 + i; // *(p + i) == arr[i];
13      }
14      // 循环结束时，p指向谁？？？ 依然指向 arr[0];
15      printf("循环结束后: p = %p\n", p);
16
17      // 使用“指针挪移”方法打印出这个数组。
18      for (size_t i = 0; i < n; i++)
19      {
20          printf("%d ", *p);
21          p++; // p = p+1, 一次加过 一个 int 大小（一个数组元素）。
22      }
23      printf("\n");
24      // 循环结束时，p指向谁？？？ 指向数组尾元素的下一个内存（野指针）。
25      printf("2 for 循环结束后: p = %p\n", p);
26
27      return 0;
28  }

```


指针 +- 整数

1. 普通指针变量 +- 整数

```
1 char *p; p+1    偏过1个字节。
2 short *p; p+1   偏过2个字节。
3 int *p; p+1     偏过4个字节。
```

2. 在数组中, +- 整数

```
1 short arr[] = {1, 3, 5, 8, 12, 17, 19};
2 short *p = arr+3;
3 p + 3;    向后（右）偏过 3 个元素。 6 个字节。
4 p - 2;    向前（左）偏过 2 个元素。 4 个字节。
```

3. &数组名 + 1

- &数组名 +1, 加过的是一个数组的总大小。

```
int main(void)
{
    short arr[] = {1,2,3,4,5,6,7,8,9,0};
    printf("arr = %p\n", arr);
    printf("&arr[0] = %p\n", &arr[0]);
    printf("arr+1 = %p\n", arr+1);

    printf("&arr    = %p\n", &arr); // 取“整个数组”的地址。
    printf("&arr+1 = %p\n", &arr + 1); // +1, 跳过一个数组 short [10]

    return 0;
}
```



选择Microsoft Visual Studio 调试控制台

```
arr = 0133FAF8
&arr[0] = 0133FAF8
arr+1 = 0133FAFA
&arr    = 0133FAF8
&arr+1 = 0133FB0C
```

指针其他运算

- 指针 * / %
 - 不允许!!!
- 指针 +- 指针
 - 指针 + 指针
 - 不允许 error!
 - 指针 - 指针。
 - 普通变量来说, 语法允许, 但, 无实际意义。
 - 对于数组来说, 得到偏移过的元素个数。

```

1  int main(void)
2  {
3      int a[] = {1,2,3,4,5,6,7,8,9,0};
4      int* p = a; // 保存数组首地址。
5
6      p = &a[3]; // 修改p保存的地址
7      printf("p-a = %d\n", p - a); // ---- 3
8
9      int* q = &a[8];
10     printf("q-p = %d\n", q - p); // ----- 5
11
12     return 0;
13 }

```

指针实现 strlen()

- 借助 数组 实现

```

1  // 数组实现
2  int myStrlen1(char str[])
3  {
4      int i = 0;
5      while (str[i] != '\0')
6      {
7          i++;
8      }
9      return i;
10 }

```

- 借助 指针 实现

```

1  // 指针实现
2  int myStrlen2(char str[])
3  {
4      char* p = str;
5      while (*p != '\0')
6      {
7          p++;
8      }
9      return p-str; // 返回元素个数。
10 }

```

指针比较运算

1. 普通变量。
 - 语法允许，但无实际意义。
2. 数组。
 - 对于数组来说，地址之间可以比大小。得到元素存储的先后顺序。

```

1  int arr[] = {1,2,3,5,6,7,8};
2  int* p = &arr[2];
3
4  if (p > arr) {
5      printf("成立\n");
6  }
7  else if (p < arr) {
8      printf("不成立");
9  }
10 else {
11     printf("==\n");
12 }

```

3. 判断 NULL

```

1  int* p;
2  int a = 10;
3  // p = NULL // 初始化空指针。
4  p = &a;
5
6  if (p != NULL)
7  {
8      printf("p is not NULL\n");
9  }
10 else
11 {
12     printf("p is NULL\n");
13 }

```

指针数组

1. 指针数组的本质，是一个二级指针。

```

1  int a = 10;
2  int b = 20;
3  int c = 30;
4
5  int *arr[] = {&a, &b, &c}; // int型指针数组，保存地址的数组。
6
7  printf("*arr[0] = %d\n", *arr[0]); // arr[0] == *(arr+0)
8  printf("*arr[0] = %d\n", *((arr + 0)));
9  printf("*arr[0] = %d\n", **arr); // * 结合性，自右向左。

```

2. 二维数组，是指针数组，是二级指针。

```
1  int a[] = { 10 };
2  int b[] = { 20 };
3  int c[] = { 30 };
4
5  int* arr[] = {a, b, c}; // 存地址，
6
7  printf("arr[0][0] = %d\n", arr[0][0]);
8  printf("*(*(arr+0)) = %d\n", (*(arr+0)));
9  printf("**arr = %d\n", **arr); // 二级指针的简介引用。
```

多级指针
